

Interfaces web rápidas

Disciplina: Programação aplicada à engenharia cartográfica

Maurício C. M. de Paulo - D.Sc.

6 de maio de 2026



- 1 Aplicações cliente/servidor
 - Conceitos de desenvolvimento web
 - Implementação de um site básico em HTML puro
 - Site com HTML + Flask
- 2 Frameworks web para protótipos
 - Jupyter + ipywidgets
 - Streamlit
- 3 Clientes web rápidos
- 4 Exercícios complementares: Serviços REST



- 1 Aplicações cliente/servidor
 - Conceitos de desenvolvimento web
 - Implementação de um site básico em HTML puro
 - Site com HTML + Flask
- 2 Frameworks web para protótipos
 - Jupyter + ipywidgets
 - Streamlit
- 3 Clientes web rápidos
- 4 Exercícios complementares: Serviços REST



- 1 Aplicações cliente/servidor
 - Conceitos de desenvolvimento web
 - Implementação de um site básico em HTML puro
 - Site com HTML + Flask
- 2 Frameworks web para protótipos
 - Jupyter + ipywidgets
 - Streamlit
- 3 Clientes web rápidos
- 4 Exercícios complementares: Serviços REST

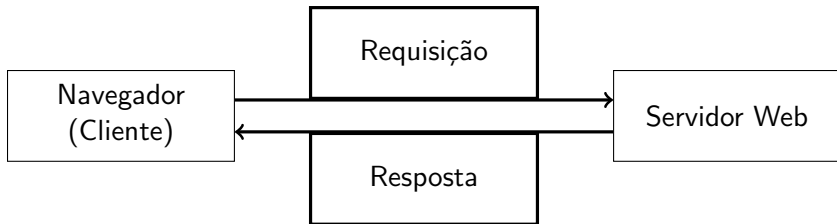
Cliente: O navegador do usuário (ex.: Chrome). Solicita recursos.

Servidor: Máquina que hospeda o site/API. Processa requisições.

Interação:

Cliente $\xrightarrow{\text{Requisição}}$ Servidor $\xrightarrow{\text{Resposta}}$ Cliente

Objetivo: Entregar conteúdo dinâmico e funcional.





Definição: Código executado no navegador do usuário.

- **HTML:** Estrutura e conteúdo (esqueleto)
- **CSS:** Estilo e layout (aparência)
- **JavaScript:** Interatividade (comportamento)

Exemplo (JS):

```
function clickHandler() {  
    console.log("Botão clicado.");  
}
```



Definição: Código executado no servidor. Implementa regras de negócio.

Componentes:

- **Linguagem:** Python, Node.js, Java, etc.
- **Framework:** Estrutura (Flask, Django)
- **Banco de dados:** Armazenamento persistente

Processo:

Requisição HTTP → processamento → consulta ao BD → resposta



Função: Armazenar e recuperar dados.

Tipos:

- **SQL:** Estruturado (PostgreSQL, MySQL)
- **NoSQL:** Flexível (MongoDB)

Conceito-chave: A API controla o acesso aos dados.



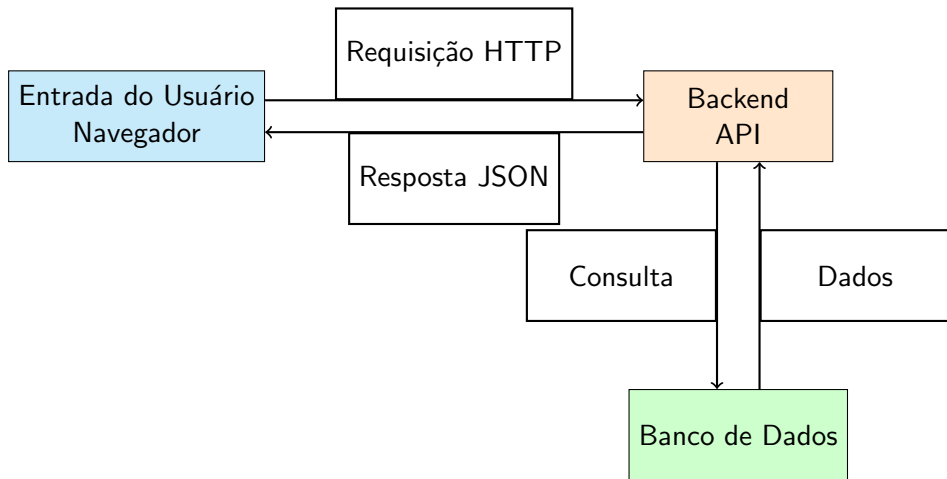
Definição: Regras de comunicação entre sistemas.

REST: Padrão baseado em HTTP.

Métodos HTTP:

- GET → leitura
- POST → criação
- PUT/PATCH → atualização
- DELETE → remoção

Formato: JSON



Cliente solicita → Servidor processa → Banco responde → Cliente recebe

Monólito: Uma única base de código gerencia todo o sistema (simples no início, difícil de escalar).

Microserviços: Aplicação dividida em pequenos serviços independentes que se comunicam.

- **Vantagem:** Independência tecnológica, alto isolamento de falhas e escalabilidade granular.

Tendência Principal: Desacoplamento

- **Frontend** ↔ **Backend** se comunicam exclusivamente via APIs.
- Permite que equipes trabalhem de forma independente no cliente e no servidor.



Área	Tecnologia Principal	Função
Lado do Cliente	HTML/CSS/JS	Apresentação/Interação
Lógica de Backend	Python/Node.js/etc.	Processamento de Requisições
Camada de Dados	SQL/NoSQL	Persistência/Armazenamento
Comunicação	API REST (JSON)	Definição de Interface



Desenvolvimento web é um sistema em camadas

Cliente → API → Backend → Banco

Entender cada camada é essencial



- 1 Aplicações cliente/servidor
 - Conceitos de desenvolvimento web
 - Implementação de um site básico em HTML puro
 - Site com HTML + Flask
- 2 Frameworks web para protótipos
 - Jupyter + ipywidgets
 - Streamlit
- 3 Clientes web rápidos
- 4 Exercícios complementares: Serviços REST

```
<!DOCTYPE html>
<html>
  <body>

    <form action="/" method="post">

      Nome:
      <input type="text" name="nome"><br><br>

      Curso:
      <select name="curso">
        <option value="python">Python</option>
        <option value="geoprocessamento">Geoprocessamento</option>
        <option value="web">Web</option>
      </select><br><br>

      <input type="submit" value="Enviar">

    </form>

  </body>
</html>
```



Objetivo: Servir arquivos locais rapidamente (HTML, imagens, etc.)

Executar no terminal:

```
python -m http.server 8000
```

Acessar no navegador:

- `http://localhost:8000`

Estrutura:

- Serve arquivos do diretório atual
- Exibe listagem automática

Observações:

- Ideal para testes rápidos
- Não usar em produção



- 1 Aplicações cliente/servidor
 - Conceitos de desenvolvimento web
 - Implementação de um site básico em HTML puro
 - Site com HTML + Flask
- 2 Frameworks web para protótipos
 - Jupyter + ipywidgets
 - Streamlit
- 3 Clientes web rápidos
- 4 Exercícios complementares: Serviços REST

Fluxo da aplicação:

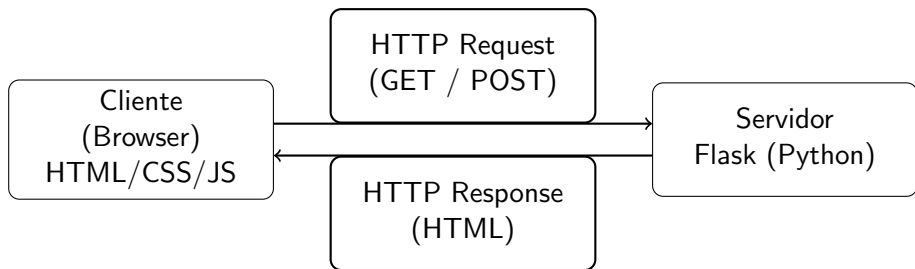
- 1 Usuário preenche formulário HTML
- 2 Navegador envia requisição HTTP (POST)
- 3 Flask processa via rota (@app.route)
- 4 Servidor retorna HTML como resposta

Flask:

- Microframework web em Python
- Baseado em WSGI

Execução:

- `pixi add flask`
- `pixi run python app.py`



Usuário interage
via navegador

Processa lógica
e retorna HTML



```
from flask import Flask, request
app = Flask(__name__)

@app.route("/", methods=["GET", "POST"])
def index():
    if request.method == "POST":
        nome = request.form.get("nome", "")
        curso = request.form.get("curso", "")
        return f"""<h2>Dados recebidos no servidor</h2>

Nome: {nome}<br>
Curso: {curso}
    """

    else: return """<form method="post">
Nome: <input type="text" name="nome"><br><br>
Curso:
<select name="curso">
    <option value="python">Python</option>
</select><br><br>
<input type="submit" value="Enviar">
</form>"""

if __name__ == "__main__":
    app.run(debug=True)
```



Isso tudo é muito bacana, mas não parece prático. Tem algo mais prático?



Isso tudo é muito bacana, mas não parece prático. Tem algo mais prático?

Backend e Frontend integrados: Streamlit



- 1 Aplicações cliente/servidor
 - Conceitos de desenvolvimento web
 - Implementação de um site básico em HTML puro
 - Site com HTML + Flask
- 2 Frameworks web para protótipos
 - Jupyter + ipywidgets
 - Streamlit
- 3 Clientes web rápidos
- 4 Exercícios complementares: Serviços REST



- Código acadêmico geralmente é:
 - Difícil de usar
 - Pouco interativo
 - Dependente de scripts ou CLI
- Dificulta:
 - Reprodutibilidade
 - Demonstração de resultados
 - Adoção por outros pesquisadores

- Criar interfaces leves para experimentação
- Tornar parâmetros ajustáveis
- Visualizar resultados em tempo real

Ferramentas: (existem outras)

- Jupyter + ipywidgets
- Streamlit
- Gradio
- Dash
- Rerun

- Jupyter:
 - + Simples e direto
 - - Pouco escalável como app
- Streamlit:
 - + Apps completos
 - + Layout flexível
- Gradio:
 - + Extremamente rápido
 - + Ideal para ML
- Dash:
 - + Muito poderoso para dashboards
 - + Baseado em Flask + React
 - - Mais verboso
- Rerun:
 - + Excelente para visualização de dados em tempo real
 - + Muito usado em robótica/visão computacional
 - - Menos focado em apps web tradicionais



- Separar lógica do modelo da interface
- Usar funções puras sempre que possível
- Controlar parâmetros via UI
- Evitar hardcode



- 1 Aplicações cliente/servidor
 - Conceitos de desenvolvimento web
 - Implementação de um site básico em HTML puro
 - Site com HTML + Flask
- 2 Frameworks web para protótipos
 - Jupyter + ipywidgets
 - Streamlit
- 3 Clientes web rápidos
- 4 Exercícios complementares: Serviços REST

- Ideal para exploração interativa
- Integra diretamente com notebooks
- Sem necessidade de frontend

```
import ipywidgets as widgets
from ipywidgets import interact

def modelo(alpha):
    return alpha * 2

interact(modelo, alpha=(0, 10))
```



- Pesquisa exploratória
- Ensino
- Prototipação rápida

Limitações:

- Difícil de compartilhar como app (voila)
- Dependência do ambiente Jupyter



- Endereço no github:
- `https://github.com/mauriciodev/progcart/blob/main/aulas/jupyter/geopandas\_jupyter\_leafmap.ipynb`
- Endereço para abrir no Google Colab:
- `https://colab.research.google.com/github/mauriciodev/progcart/blob/main/aulas/jupyter/geopandas\_jupyter\_leafmap.ipynb`
- Caderno de dados espaciais em Jupyter
Abra no colab
- Caderno IPyWidgets
Abra no colab

```
pixi add install ipywidgets
```

Ativar (se necessário):

```
jupyter nbextension enable --py widgetsnbextension
```


Slider (IntSlider)

```
import ipywidgets as widgets
from IPython.display import display

widgets.IntSlider(
    value=10,
    min=0,
    max=100,
    step=1,
    description='Valor:'
)
```

FloatSlider

```
widgets.FloatSlider(
    value=0.5,
    min=0.0,
    max=1.0,
    step=0.01,
    description='Float:'
)
```

Checkbox

```
widgets.Checkbox(  
    value=True,  
    description='Ativar'  
)
```

Dropdown

```
widgets.Dropdown(  
    options=['A', 'B', 'C'],  
    value='A',  
    description='Escolha:'  
)
```

RadioButtons

```
widgets.RadioButton(  
    options=['Opção 1', 'Opção 2'],  
    description='Opções:'  
)
```

SelectMultiple

```
widgets.SelectMultiple(  
    options=['A', 'B', 'C'],  
    description='Múltiplo:'  
)
```

Text

```
widgets.Text(  
    placeholder='Digite algo',  
    description='Texto:'  
)
```

Textarea

```
widgets.Textarea(  
    placeholder='Texto longo',  
    description='Área:'  
)
```

Button

```
button = widgets.Button(description="Clique")

def on_click(b):
    print("Clicado!")

button.on_click(on_click)
button
```

DatePicker

```
widgets.DatePicker(
    description='Data'
)
```

ColorPicker

```
widgets.ColorPicker(
    description='Cor'
)
```



```
widgets.HBox([
    widgets.Button(description="A"),
    widgets.Button(description="B")
])

widgets.VBox([
    widgets.IntSlider(),
    widgets.Text()
])
```



Objetivo: Transformar um notebook Jupyter em aplicação web interativa.

1. Instalação:

```
pixi add voila
```

2. Executar o Voila:

```
voila seu_notebook.ipynb
```

3. Resultado:

- Remove células de código
- Exibe apenas outputs e widgets
- Abre no navegador automaticamente

5. Deploy (exemplo):

- Binder
- Heroku
- Servidor próprio



- 1 Aplicações cliente/servidor
 - Conceitos de desenvolvimento web
 - Implementação de um site básico em HTML puro
 - Site com HTML + Flask
- 2 Frameworks web para protótipos
 - Jupyter + ipywidgets
 - Streamlit
- 3 Clientes web rápidos
- 4 Exercícios complementares: Serviços REST



- Framework Python para aplicações web interativas
- Foco em simplicidade e prototipação rápida
- Executa com:

```
pixi shell  
pixi add streamlit  
streamlit run app.py
```

Principais conceitos:

- Layout declarativo
- Widgets interativos
- Atualização automática da interface

Documentação ([link](#))

- Cria apps web com Python puro
- Atualização automática ao alterar código
- Muito usado em ciência de dados

```
import streamlit as st

x = st.slider("Valor", 0, 10)
st.write("Resultado:", x * 2)
```



```
import streamlit as st
import matplotlib.pyplot as plt
import leafmap.foliummap as leafmap

st.title("Exemplo Streamlit + Matplotlib + Leafmap")

st.sidebar.header("Controles")

lat = st.sidebar.number_input("Latitude", value=-15.78)
lon = st.sidebar.number_input("Longitude", value=-47.93)
zoom_btn = st.sidebar.button("Zoom na coordenada")
```

- Sidebar contém widgets
- Valores são reavaliados a cada interação



```
st.subheader("Gráfico Matplotlib")

fig, ax = plt.subplots()
ax.plot([1, 2, 3, 4], [10, 20, 25, 30])
ax.set_title("Exemplo de gráfico")

st.pyplot(fig)
```

- Criamos a figura normalmente
- Usamos `st.pyplot()` para renderizar



```
st.subheader("Mapa Leafmap")

m = leafmap.Map(center=[lat, lon], zoom=4)

if zoom_btn:
    m.set_center(lon, lat, zoom=10)

m.add_marker(location=[lat, lon],
              popup="Coordenada selecionada")

m.to_streamlit(height=500)
```

- Leafmap usa base Folium
- Botão altera o centro do mapa



```
import streamlit as st

if "users" not in st.session_state: # Estado persistente
    st.session_state.users = [ {"id": 1, "name": "Ana"} ]

col1, col2 = st.columns(2) # Layout em duas colunas

with col1: # Coluna 1: Entrada
    st.subheader("Adicionar usuário")
    name = st.text_input("Nome")

    if st.button("Adicionar"):
        if name:
            new_user = {
                "id": len(st.session_state.users) + 1,
                "name": name
            }
            st.session_state.users.append(new_user)
            st.success(f"{name} adicionado!")

with col2: # Coluna 2: Visualização
    st.subheader("Lista de usuários")
    st.write(st.session_state.users)
```



Entrada de Texto

```
import streamlit as st

name = st.text_input("Nome")
st.write("Olá,", name)
```

Número e Slider

```
x = st.number_input("Valor", min_value=0, max_value=10)

y = st.slider("Escolha um valor", 0, 100, 50)

st.write(x, y)
```

Checkbox e Radio

```
flag = st.checkbox("Ativar")

opcao = st.radio(
    "Escolha uma opção",
    ["A", "B", "C"]
)

st.write(flag, opcao)
```

Selectbox e Multiselect

```
item = st.selectbox(
    "Escolha um item",
    ["Python", "R", "Julia"]
)

itens = st.multiselect(
    "Escolha vários",
    ["A", "B", "C"]
)

st.write(item, itens)
```



Botão

```
if st.button("Clique aqui"):  
    st.write("Botão pressionado!")
```

Upload de Arquivo

```
file = st.file_uploader("Envie um arquivo")  
  
if file:  
    st.write(file.name)
```

Data e Cor

```
date = st.date_input("Data")  
  
color = st.color_picker("Escolha uma cor")  
  
st.write(date, color)
```




Exibição de Dados

```
import pandas as pd

df = pd.DataFrame({
    "A": [1,2,3],
    "B": [4,5,6]
})

st.dataframe(df)
st.table(df)
```



- Controla organização da interface
- Permite criar apps mais estruturados

Container

```
container = st.container()

with container:
    st.write("Conteúdo agrupado")
```



Colunas

```
col1, col2 = st.columns(2)

with col1:
    st.write("Coluna 1")

with col2:
    st.write("Coluna 2")
```

Sidebar

```
st.sidebar.title("Menu")

op = st.sidebar.selectbox(
    "Escolha",
    ["Home", "Dados"]
)
```



Expander

```
with st.expander("Detalhes"):  
    st.write("Informações escondidas")
```

Abas

```
tab1, tab2 = st.tabs(["Aba 1", "Aba 2"])  
  
with tab1:  
    st.write("Conteúdo 1")  
  
with tab2:  
    st.write("Conteúdo 2")
```



Dash <https://dash.plotly.com/>
Rerun <https://rerun.io/examples>
Gradio: <https://www.gradio.app/>



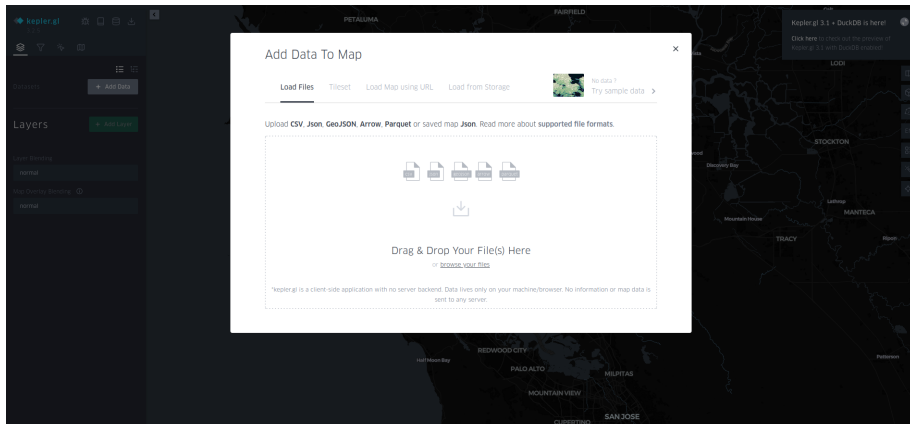
- 1 Aplicações cliente/servidor
 - Conceitos de desenvolvimento web
 - Implementação de um site básico em HTML puro
 - Site com HTML + Flask
- 2 Frameworks web para protótipos
 - Jupyter + ipywidgets
 - Streamlit
- 3 Clientes web rápidos
- 4 Exercícios complementares: Serviços REST

`https://kepler.gl/`

Pode ser gerado pela Leafmap

Trabalho de alunos do 2o ano 2025:

`https://mauriciodev.github.io/progcart/ipe2.html`





- Interfaces aumentam o impacto do código acadêmico
- Reduzem barreiras de uso
- Facilitam validação e colaboração



- 1 Aplicações cliente/servidor
 - Conceitos de desenvolvimento web
 - Implementação de um site básico em HTML puro
 - Site com HTML + Flask
- 2 Frameworks web para protótipos
 - Jupyter + ipywidgets
 - Streamlit
- 3 Clientes web rápidos
- 4 Exercícios complementares: Serviços REST



- Criar uma API REST simples
- Testar com cliente Python
- Visualizar no navegador
- Documentar com Swagger

```
from flask import Flask, jsonify

app = Flask(__name__)

@app.route("/users")
def get_users():
    return jsonify([
        {"id": 1, "name": "Ana"},
        {"id": 2, "name": "Bruno"}
    ])

if __name__ == "__main__":
    app.run(debug=True)
```

Executar:

```
python app.py
```



- Acesse no navegador:
- `http://127.0.0.1:5000/users`

Resultado:

- JSON exibido diretamente
- Fácil inspeção
- Útil para debug rápido



```
import requests

url = "http://127.0.0.1:5000/users"

response = requests.get(url)

print("Status:", response.status_code)
print("JSON:", response.json())
```

Principais métodos HTTP:

- **GET** → Buscar dados (sem alterar estado)
- **POST** → Enviar dados / criar recurso
- **PUT** → Atualizar recurso (completo)
- **PATCH** → Atualizar parcialmente
- **DELETE** → Remover recurso

Exemplos em Python:

```
import requests

# GET
r = requests.get("http://api.exemplo.com/users")

# POST
r = requests.post( "http://api.exemplo.com/users", json={"name": "Ana"} )

# PUT
r = requests.put( "http://api.exemplo.com/users/1", json={"name": "Ana Silva"} )

# DELETE
r = requests.delete( "http://api.exemplo.com/users/1" )
```

Instalar:

```
pixi add flasgger
```

Adicionar ao Flask:

```
from flasgger import Swagger

app = Flask(__name__)
swagger = Swagger(app)

@app.route("/users")
def get_users():
    """
    Lista de usuários
    ---
    responses:
      200:
        description: Lista de usuários
    """
    return jsonify([
        {"id": 1, "name": "Ana"}
    ])
```

- Acesse:
- `http://127.0.0.1:5000/apidocs`

Vantagens:

- Interface interativa
- Teste direto no navegador
- Documentação automática



- Flask → fornece API
- Browser → visualiza JSON
- Requests → cliente Python
- Swagger → documentação + teste

Pipeline didático:

- Implementar → Testar → Visualizar → Documentar

Inserindo Dados (POST)



```
from flask import Flask, jsonify, request
app = Flask(__name__)

# "Banco" em memória. Deve ser trocado para outro tipo de persistência de dados.
users = [
    {"id": 1, "name": "Ana"}
]

@app.route("/users", methods=["GET"])
def get_users():
    return jsonify(users)

@app.route("/users", methods=["POST"])
def add_user():
    data = request.get_json()
    new_user = {
        "id": len(users) + 1,
        "name": data.get("name")
    }
    users.append(new_user)
    return jsonify(new_user), 201

app.run(debug=True)
```